21.4 Web Application Testing

Many Web testing practices are also appropriate for testing thin-client MobileApps and interactive simulations. The strategy for WebApp testing adopts the basic principles for all software testing and applies a strategy and tactics that are used for object-oriented systems. The following steps summarize the approach:

1. The content model for the WebApp is reviewed to uncover errors.
2. The interface model is reviewed to ensure that all use cases can be accommodated.
3. The design model for the WebApp is reviewed to uncover navigation errors.
4. The user interface is tested to uncover errors in presentation and/or navigation mechanics.
5. Each functional component is unit tested.
6. Navigation throughout the architecture is tested.
7. The WebApp is implemented in a variety of different environmental configurations and is tested for compatibility with each configuration.
8. Security tests are conducted in an attempt to exploit vulnerabilities in the WebApp or within its environment.
9. Performance tests are conducted.
10. The WebApp is tested by a controlled and monitored population of end users. The results of their interaction with the system are evaluated for errors.

Because many WebApps evolve continuously, the testing process is an ongoing activity, conducted by support staff who use regression tests derived from the tests developed when the WebApp was first engineered. Methods for WebApp testing are considered in Section 21.5.

21.5 Web Testing Strategies

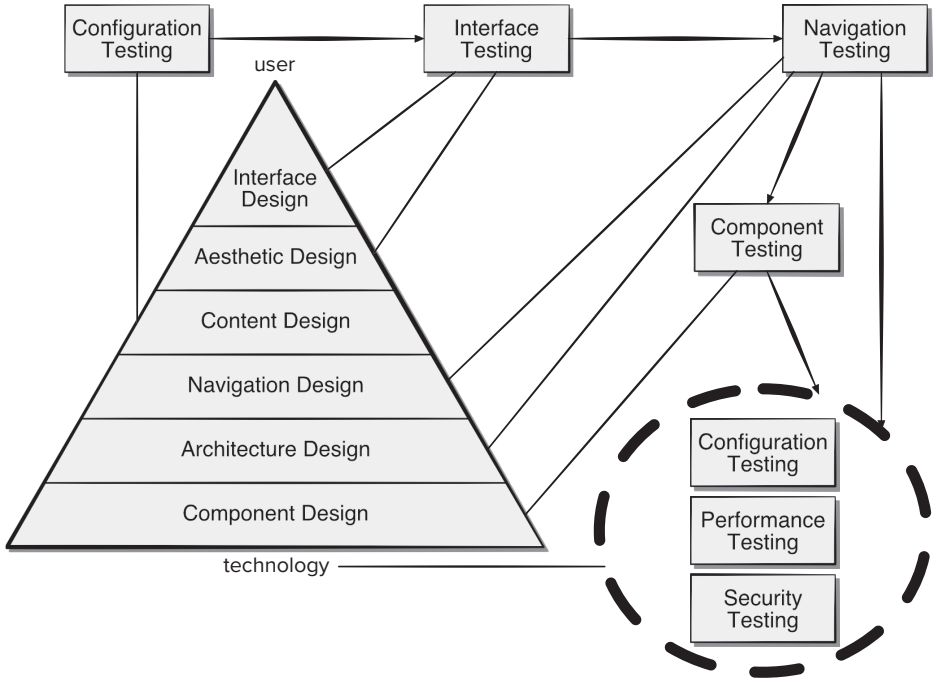
Testing is the process of exercising software with the intent of finding (and ultimately correcting) errors. This fundamental philosophy, first presented in Chapter 20, does not change for WebApps. In fact, because Web-based systems and applications reside on a network and interoperate with many different operating systems, browsers (or other personal communication devices), hardware platforms, communications protocols, and "backroom" applications, the search for errors represents a significant challenge.

Figure 21.2 juxtaposes the mobility testing process with the design pyramid for WebApps (Chapter 13). Note that as the testing flow proceeds from left to right and top to bottom, user-visible elements of the WebApp design (top elements of the pyramid) are tested first, followed by infrastructure design elements.

Because many apps evolve continuously, the testing process is an ongoing activity, conducted by app support staff who use regression tests derived from the tests developed when the app was first engineered.

FIGURE 21.2

The testing process



SAFEHOME



WebApp Testing

The scene: Doug Miller's office.

The players: Doug Miller, manager of the *SafeHome* software engineering group, and Vinod Raman, a member of the product software engineering team.

The conversation:

Doug: What do you think of the **SafeHome Assured.com** e-commerce WebApp V0.0?

Vinod: The outsourcing vendor has done a good job. Sharon [development manager for the vendor] tells me they're testing as we speak.

Doug: I'd like you and the rest of the team to do a little informal testing on the e-commerce site.

Vinod (grimacing): I thought we were going to hire a third-party testing company to validate the WebApp. We're still killing ourselves trying to get the product software out the door.

Doug: We're going to hire a testing vendor for performance and security testing, and our outsourcing vendor is already testing. Just thought another point of view would be helpful, and besides, we'd like to keep costs in line, so . . .

Vinod (sighs): What are you looking for?

Doug: I want to be sure that the interface and all navigation are solid.

Vinod: I suppose we can start with the use cases for each of the major interface functions:

Learn about *SafeHome*.
Specify the *SafeHome* system you need.
Purchase a *SafeHome* system.
Get technical support.

Doug: Good. But take the navigation paths all the way to their conclusion.

Vinod (looking through a notebook of use cases): Yeah, when you select **Specify the SafeHome system you need**, that'll take you to:

Select SafeHome components.
Get *SafeHome* component recommendations.

We can exercise the semantics of each path.

Doug: While you're there, check out the content that appears at each navigation node.

Vinod: Of course . . . and the functional elements as well. Who's testing usability?

Doug: Oh . . . the testing vendor will coordinate usability testing. We've hired a market research firm to line up 20 typical users for the usability study, but if you guys uncover any usability issues . . .

Vinod: I know, pass them along.

Doug: Thanks, Vinod.

21.5.1 Content Testing

Errors in WebApp content can be as trivial as minor typographical mistakes or as significant as incorrect information, improper organization, or violation of intellectual property laws. *Content testing* attempts to uncover these and many other problems before the user encounters them.

Content testing has three important objectives: (1) to uncover syntactic errors (e.g., typos, grammar mistakes) in text-based documents, graphical representations, and other media; (2) to uncover semantic errors (i.e., errors in the accuracy or completeness of information) in any content object presented as navigation occurs; and (3) to find errors in the organization or structure of content that is presented to the end user.

Content testing combines both reviews and the generation of executable test cases. Although technical reviews are not a part of testing, content review should be performed to ensure that content has quality and to uncover semantic errors. Executable testing is used to uncover content errors that can be traced to dynamically derived content that is driven by data acquired from one or more databases.

To accomplish the first objective, automated spelling and grammar checkers may be used. However, many syntactic errors evade detection by such tools and must be discovered by a human reviewer (tester). In fact, a large website might enlist the services of a professional copy editor to uncover typographical errors, grammatical mistakes, errors in content consistency, errors in graphical representations, and cross-referencing errors.

Semantic testing focuses on the information presented within each content object. The reviewer (tester) must answer the following questions:

- Is the information factually accurate?
- Is the information concise and to the point?
- Is the layout of the content object easy for the user to understand?
- Can information embedded within a content object be found easily?
- Have proper references been provided for all information derived from other sources?
- Is the information presented consistent internally and consistent with information presented in other content objects?

- Is the content offensive, misleading, or does it open the door to litigation?
- Does the content infringe on existing copyrights or trademarks?
- Does the content contain internal links that supplement existing content? Are the links correct?
- Does the aesthetic style of the content conflict with the aesthetic style of the interface?

Obtaining answers to each of these questions for a large WebApp (containing hundreds of content objects) can be a daunting task. However, failure to uncover semantic errors will shake the user's faith in the WebApp and can lead to failure of the Web-based application.

21.5.2 Interface Testing

Interface testing exercises interaction mechanisms and validates aesthetic aspects of the user interface. The overall strategy for interface testing is to (1) uncover errors related to specific interface mechanisms (e.g., errors in the proper execution of a menu link or the way data are entered in a form) and (2) uncover errors in the way the interface implements the semantics of navigation, WebApp functionality, or content display. With the exception of WebApp-oriented specifics, the interface strategy noted here is applicable to all types of client-server software. To accomplish this strategy, a number of tactical steps are initiated:

- Interface features are tested to ensure that design rules, aesthetics, and related visual content are available for the user without error.
- Individual interface mechanisms are tested in a manner that is analogous to unit testing. For example, tests are designed to exercise all forms, client-side scripting, dynamic HTML, scripts, streaming content, and application-specific interface mechanisms (e.g., a shopping cart for an e-commerce application).
- Each interface mechanism is tested within the context of a use case or network semantic unit (NSU) (Chapter 13) for a specific user category.
- The complete interface is tested against selected use cases and NSUs to uncover errors in the semantics of the interface. It is at this stage that a series of usability tests are conducted.
- The interface is tested within a variety of environments (e.g., browsers) to ensure that it will be compatible.

21.5.3 Navigation Testing

A user travels through a WebApp in much the same way as a visitor walks through a store or museum. There are many pathways to take, stops to make, things to learn and look at, activities to initiate, and decisions to make. This navigation process is predictable in the sense that every visitor has a set of objectives when he arrives. At the same time, the navigation process can be unpredictable because the visitor, influenced by something she sees or learns, may choose a path or initiate an action that is not typical for the original objective. The job of navigation testing is (1) to ensure that the mechanisms that allow the WebApp user to travel through the

WebApp are all functional and (2) to validate that each NSU can be achieved by the appropriate user category.

The first phase of navigation testing actually begins during interface testing. Navigation mechanisms (links and anchors of all types, redirects,[2] bookmarks, frames and frame sets, site maps, and the accuracy of internal search facilities) are tested to ensure that each performs its intended function. Some of the tests noted can be performed by automated tools (e.g., link checking), while others are designed and executed manually. The intent throughout is to ensure that errors in navigation mechanics are found before the WebApp goes online.

Each NSU (Chapter 13) is defined by a set of navigation paths (called "the user journey") that connect navigation nodes (e.g., Web pages, content objects, or functionality). Taken as a whole, each NSU allows a user to achieve specific requirements defined by one or more use cases for a user category. Navigation testing exercises each NSU to ensure that these requirements can be achieved. If NSUs have not been created as part of WebApp analysis or design, you can apply use cases for the design of navigation test cases. You should answer the following questions as each NSU or use case is tested:

- Is the NSU achieved in its entirety without error?
- Is every navigation node (defined for an NSU) reachable within the context of the navigation paths defined for the NSU?
- If the NSU can be achieved using more than one navigation path, has every relevant path been tested?
- If guidance is provided by the user interface to assist in navigation, are directions correct and understandable as navigation proceeds?
- Is there a mechanism (other than the browser "back" arrow) for returning to the preceding navigation node and to the beginning of the navigation path?
- Do mechanisms for navigation within a large navigation node (i.e., a long Web page) work properly?
- If a function is to be executed at a node and the user chooses not to provide input, can the remainder of the NSU be completed?
- If a function is executed at a node and an error in function processing occurs, can the NSU be completed?
- Is there a way to discontinue the navigation before all nodes have been reached, but then return to where the navigation was discontinued and proceed from there?
- Is every node reachable from the site map? Are node names meaningful to end users?
- If a node within an NSU is reached from some external source, is it possible to process to the next node on the navigation path? Is it possible to return to the previous node on the navigation path?
- Does the user understand his location within the content architecture as the NSU is executed?

2 When a server request is forwarded to a nonexistent URL.

Navigation testing, like interface and usability testing, should be conducted by as many different constituencies as possible. You have responsibility for early stages of navigation testing, but later stages should be conducted by other project stakeholders, an independent testing team, and ultimately, by nontechnical users. The intent is to exercise WebApp navigation thoroughly.

21.6 INTERNATIONALIZATION

Internationalization is the process of creating a software product so that it can be used in several countries and with various languages without requiring any engineering changes. *Localization* is the process of adapting a software application for use in targeted global regions by adding locale-specific requirements and translating text elements to appropriate languages. Localization effort may involve taking each country's currency, culture, taxes, and standards (both technical and legal) into account in addition to differences in languages [Sla12]. Launching a MobileApp in many parts of the world without testing it there would be very foolish.

Because it can be very costly to build an in-house testing facility in each country for which localization is planned, outsourcing testing to local vendors in each country is often more cost effective [Reu12]. However, using an outsourcing approach risks a degradation of communication between the MobileApp development team and those who are performing localization tests.

Crowdsourcing has become popular in many online communities.[3] Reuveni [Reu12] suggests that crowdsourcing could be used to engage localization testers dispersed around the globe outside of the development environment. To accomplish this, it is important to find a community that prides itself on its reputation and has a track record of successes. An easy-to-use real-time platform allows community members to communicate with the project decision makers. To protect intellectual property, only trustworthy community members who are willing to sign nondisclosure agreements are allowed to participate.

21.7 SECURITY TESTING

Any computer-based system that manages sensitive information or causes actions that can improperly harm (or benefit) individuals is a target for improper or illegal penetration. Penetration spans a broad range of activities: hackers who attempt to penetrate systems for sport, disgruntled employees who attempt to penetrate for revenge, dishonest individuals who attempt to penetrate for illicit personal gain.

Security testing attempts to verify that protection mechanisms built into a system will, in fact, protect it from improper penetration. Given enough time and resources, thorough security testing will ultimately penetrate a system. The role of the system designer is to make penetration cost more than the value of the information that will

3 *Crowdsourcing* is a distributed problem-solving model where community members work on solutions to problems posted to the group.